CodInClub
Powered By BridgeLabz

Best Practices in Constructors

1. **Use `this` Keyword**:
    - Avoid ambiguity when parameter names are the same as attribute names.
    - Example: `this.customerName = customerName;`
2. **Keep Logic Simple**:
    - Avoid heavy computations or database calls inside constructors.
3. **Provide Multiple Constructors**:
    - Support various initialization scenarios by overloading constructors.
4. **Encapsulate Logic**:
    - Use private methods (like `calculatePrice()`) to keep constructors clean.

Best Practices in Access Modifiers

Use the Least Privilege:

- Start with the most restrictive modifier (`private`) and relax it as needed (`protected` or `public`).

Encapsulation:

- Always make attributes `private` and use getters/setters for controlled access.

Protected Usage:

- Use `protected` only when inheritance is required and controlled access is necessary.

Avoid Overexposure:

- Limit the use of `public` to methods or classes that are meant to be accessed by external code.

Package Access:

- Use the default (package-private) modifier to restrict access to the same package unless explicitly needed elsewhere.

Avoid Leaks:

- Be cautious with exposing mutable objects, like collections, via getters. Return a copy or an unmodifiable view when possible.

Level 1 Practice Programs

1. Create a `Book` class with attributes `title`, `author`, and `price`. Provide both default and parameterized constructors.

```
import java.util.Scanner;
class Book {
   private String title;
   private String author;
   private double price;

   //default constructor
   public Book() {
       this.title = "unknown";
       this.author = "unknown";
       this.price = 0;
   }
   //parameterized constructor
   public Book(String title, String author, double
price) {
       this.title = title;
       this.author = author;
       this.price = price;
   }
```

```
    //method to print book details
    public void printDetails() {
        System.out.println("Title : " + title);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}
public class BookDetails2 {
    public static void main(String[] args) {

        //creating scanner object
        Scanner input = new Scanner(System.in);

       //creating object book1 of book class using
default constructor
        Book book1 = new Book();

        //taking user inputs for title author and price
for book2 defined below
        System.out.print("Enter title : ");
        String title = input.nextLine();
        System.out.print("Enter author : ");
        String author = input.nextLine();
        System.out.print("Enter price : ");
        double price = input.nextDouble();

        //creating object  book2 of book class using
parameterized constructor
        Book book2 = new Book(title, author, price);
```

```
        //printing details of book1 object
        System.out.println("Details of book1 object :
");
        book1.printDetails();

        //printing details of book2 object
        System.out.println("Details of book2 object :
");
        book2.printDetails();

        //closing input
        input.close();
    }
}
```

2. Write a `Circle` class with a `radius` attribute. Use constructor chaining to initialize `radius` with default and user-provided values.

```
import java.util.Scanner;
class Circle {
    private final double PI = 3.14;
    private double radius;
    private double area;
    private double circumference;

    //making default constructor
```

```
    public Circle() {
        //declaring radius by default zero if user not
inserted
        this(0.0);
    }
    //parameterized constructor
    public Circle(double radius) {
        this.radius = radius;
    }

    //method to calculate area and circumference
    public void calculateAreaAndCircumference() {

        //calculating area
        area = PI * radius * radius;

        //calculating circumference
        circumference = 2 * PI * radius;
    }


    //method to print area and circumference
    public void printAreaAndCircumference() {
        System.out.println("Area : " + area);
        System.out.println("Circumference : " +
circumference);
    }
}
public class CircleDetails2 {
```

```
   public static void main(String[] args) {

        //creating scanner object
        Scanner input = new Scanner(System.in);

        //taking user input for radius
        System.out.print("Enter radius : ");
        double radius = input.nextDouble();

        //creating object of circle class using default
constructor
        Circle cir = new Circle();

        //creating another object of circle class using
parametrized constructor
        Circle cir2 = new Circle(radius);

        //calculating area and circumference for cir1
        cir.calculateAreaAndCircumference();

        //printing area and circumference of circle of
cir1
        cir.printAreaAndCircumference();

        //calculating area and circumference for cir2
        cir2.calculateAreaAndCircumference();

        //printing area and circumference of circle of
cir2
```

```
        cir2.printAreaAndCircumference();

        //closing input
        input.close();
    }
}
```

3. Create a `Person` class with a copy constructor that clones another person's attributes.e

```
import java.util.Scanner;
public class Person {
    private String name;
    private int age;

    //creating parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    //creating copy constructor
    public Person(Person p1) {
        this.name = p1.name;
        this.age = p1.age;
    }
    //method to print details
    public void printDetails() {
        System.out.println("Name : " + this.name);
```

```
        System.out.println("Age : " + this.age);
    }

    //main method
    public static void main(String[] args) {

        //creating scanner class object to take input
        Scanner input = new Scanner(System.in);

        //taking user input for name and age
        System.out.print("Enter Name : ");
        String name = input.nextLine();
        System.out.print("Enter age : ");
        int age = input.nextInt();

        //creating object of person class
        Person p1 = new Person(name, age);

        //creating another object using copy constructor
        Person p2 = new Person(p1);

        //printing the details of p1
        System.out.println("printing the details of p1 :
");
        p1.printDetails();
        //printing the details of p2
        System.out.println("printing the details of p2 :
");
        p2.printDetails();
```

```
        //closing the input
        input.close();
    }
}
```

4. **Hotel Booking System**: Create a `HotelBooking` class with attributes `guestName`, `roomType`, and `nights`. Use default, parameterized, and copy constructors to initialize bookings.

```
import java.util.Scanner;
public class HotelBooking {
    private String guestName;
    private String roomType;
    private int nights;

    //default constructor
    public HotelBooking() {
        this("unknown", "default", 0);
    }

    //parameterized constructor
    public HotelBooking(String guestName, String
roomType, int nights) {
        this.guestName = guestName;
        this.roomType = roomType;
        this.nights = nights;
    }
```

```
    //copy constructor
    public HotelBooking(HotelBooking guest) {
        this.guestName = guest.guestName;
        this.roomType = guest.roomType;
        this.nights = guest.nights;
    }

    //printing the details
    public void printDetails() {
        System.out.println("Guest name : " +
this.guestName);
        System.out.println("Room type : " +
this.roomType);
        System.out.println("Nights : " + this.nights);
    }
    //main class
    public static void main(String[] args) {
        //creating scanner class object
        Scanner input = new Scanner(System.in);

        //taking user input for guest name, room type
and nights
        System.out.print("Enter guest name : ");
        String guestName = input.nextLine();
        System.out.print("Enter room type : ");
        String roomType = input.nextLine();
        System.out.print("Enter number of nights : ");
        int nights = input.nextInt();
```

```
        //creating object using default constructor
        HotelBooking guest1 = new HotelBooking();

        //creating object using parameterized
constructor
        HotelBooking guest2 = new
HotelBooking(guestName, roomType, nights);

        //creating object using copy constructor
        HotelBooking guest3 = new HotelBooking(guest2);

        //displaying the details of guest 1
        System.out.println("Details of guest 1 :");
        guest1.printDetails();

        //displaying the details of guest 2
        System.out.println("Details of guest 2 :");
        guest2.printDetails();

        //displaying the details of guest 3
        System.out.println("Details of guest 3 :");
        guest3.printDetails();

        //closing the input
        input.close();
    }
}
```

5. **Library Book System**: Create a `Book` class with attributes `title`, `author`, `price`, and `availability`. Implement a method to borrow a book.

```
import java.util.Scanner;
class Book2 {
   private String title;
   private String author;
   private double price;
   private boolean availability;

   //making constructor
   public Book2(String title, String author, double price) {
       this.title = title;
       this.author = author;
       this.price = price;
       this.availability = true;
   }
   //method to borrow a book
   public void borrow() {
       this.availability = false;
   }

   //method to print book details
   public void printDetails() {
       System.out.println("Title : " + title);
       System.out.println("Author : " + author);
       System.out.println("Price : " + price);
       System.out.println("Is the book available? " + availability);
   }
}
public class Library {
   public static void main(String[] args) {

       //creating scanner object
       Scanner input = new Scanner(System.in);

       //taking user inputs for title, author and price of book
       System.out.print("Enter title of book : ");
       String title = input.nextLine();
```

```
        System.out.print("Enter author's name : ");
        String author = input.nextLine();
        System.out.print("Enter price of book : ");
        double price = input.nextDouble();

        //creating a book object
        Book2 book = new Book2(title, author, price);

        //printing details of books before borrow
        System.out.println("Details before borrow : ");
        book.printDetails();

        //borrowing book
        book.borrow();

        //printing details of books after borrow
        System.out.println("Details after borrow : ");
        book.printDetails();

        //closing input
        input.close();
    }
}
```

6. **Car Rental System**: Create a `CarRental` class with attributes `customerName`, `carModel`, and `rentalDays`. Add constructors to initialize the rental details and calculate total cost.

```
import java.util.Scanner;
public class CarRentalSystem {
    //creating data members
    private String customerName;
    private String carModel;
    private int rentalDays;
    private double cost;
```

```
    private double totalCost;

    //creating constructor
    public CarRentalSystem(String customerName, String
carModel, int rentalDays, double cost) {
        this.customerName = customerName;
        this.carModel = carModel;
        this.rentalDays = rentalDays;
        this.cost = cost;

        //calculating the total cost
        this.totalCost = rentalDays * cost;
    }

    //creating method to display details
    public void printDetails() {
        System.out.println("Name of customer : " +
this.customerName);
        System.out.println("car model : " +
this.carModel);
        System.out.println("rental days : " +
this.rentalDays);
        System.out.println("cost per day : " +
this.cost);
        System.out.println("Total cost : " +
this.totalCost);
    }
    //main method
    public static void main(String[] args) {
```

```
        //creating scanner object
        Scanner input = new Scanner(System.in);



        //taking user inputs for customerName, carModel,
rentalDays and cost per day
        System.out.print("Enter customer name : ");
        String customerName = input.nextLine();
        System.out.print("Enter car model : ");
        String carModel = input.nextLine();
        System.out.print("Enter rental days : ");
        int rentalDays = input.nextInt();
        System.out.print("Enter cost per day : ");
        double cost = input.nextDouble();

        //creating object car1 of CarRentalSystem class
        CarRentalSystem car1 = new
CarRentalSystem(customerName, carModel, rentalDays,
cost);

        //printing details of car1 object
        System.out.println("\nDetails of car1 object :
");
        car1.printDetails();

        //closing the input
        input.close();
    }
```

```
}
```

CodInClub

Powered By BridgeLabz

1. Instance vs. Class Variables and Methods

Problem 1: Product Inventory

Create a `Product` class with:

- Instance Variables: `productName`, `price`.
- Class Variable: `totalProducts` (shared among all products).
- Methods:
    - An instance method `displayProductDetails()` to display the details of a product.
    - A class method `displayTotalProducts()` to show the total number of products created.

```
        input.nextLine();
```

Problem 2: Online Course Management

Design a `Course` class with:

- Instance Variables: `courseName`, `duration`, `fee`.
- Class Variable: `instituteName` (common for all courses).
- Methods:
    - An instance method `displayCourseDetails()` to display the course details.
    - A class method `updateInstituteName()` to modify the institute name for all courses.

```
import java.util.Scanner;
```

```
public class Course {
   //creating data members;
   private String courseName;
   private int duration;
   private double fees;

   //creating class variable InstituteName to indicate
name of institute
   private static String instituteName;

   //creating constructor to initialize variables
   public Course(String courseName, int duration,
double fees) {
       this.courseName = courseName;
       this.duration = duration;
       this.fees = fees;
   }
   //method to update institute name
   public static void updateInstitute(String institute)
{
       instituteName = institute;
   }

   //method to display course details
   public void displayDetails() {
       System.out.println("course name : " +
this.courseName);
       System.out.println("Duration (in months) : " +
this.duration);
```

```
        System.out.println("Fees : " + this.fees);
        System.out.println("Institute name : " +
instituteName);
    }

    //main method
    public static void main(String[] args) {

        //creating scanner class object to take user
input
        Scanner input = new Scanner(System.in);

        //taking user input for institute name
        System.out.print("Enter institute name : ");
        String instituteName = input.nextLine();

        //updating the institute name
        updateInstitute(instituteName);

        //taking user input for course details
        System.out.println("Enter details for course " +
1);
        System.out.print("Enter course name : ");
        String courseName = input.nextLine();
        System.out.print("Enter Duration (in months) :
");
        int duration = input.nextInt();
        System.out.print("Enter fees : ");
        double fees = input.nextDouble();
```

```
        //creating object course1 of Product class
        Course course1 = new Course(courseName,
duration, fees);

        //printing details of course 1
        System.out.println("details of course 1 : ");
        course1.displayDetails();

        input.nextLine();
        //taking user input for new institute name
        System.out.print("Enter new institute name : ");
        instituteName = input.nextLine();

        //updating the institute name
        updateInstitute(instituteName);

        //taking user input for course details
        System.out.println("Enter details for course " +
2);
        System.out.print("Enter course name : ");
        courseName = input.nextLine();
        System.out.print("Enter Duration (in months) :
");
        duration = input.nextInt();
        System.out.print("Enter fees : ");
        fees = input.nextDouble();

        //creating object course1 of Course class
```

```
        Course course2 = new Course(courseName,
duration, fees);

        //printing details of course 2
        System.out.println("details of course 2 : ");
        course2.displayDetails();

        //printing details of course 1 and you can see
institute name is also updated for course 1
        System.out.println("details of course 1 : ");
        course1.displayDetails();

        System.out.println("you can see above the
institute name is also updated for course 1");

        //closing the input
        input.close();
    }
}
```

Problem 3: Vehicle Registration

Create a `Vehicle` class to manage the details of vehicles:

- Instance Variables: `ownerName`, `vehicleType`.
- Class Variable: `registrationFee` (fixed for all vehicles).
- Methods:
    - An instance method `displayVehicleDetails()` to display owner and vehicle details.

- A class method `updateRegistrationFee()` to change the registration fee.

```
import java.util.Scanner;
public class Vehicle {
   //creating data members
   private String ownerName;
   private String vehicleName;

   //creating class variable registrationFee
   private static double registrationFee;

   //creating constructor to initialize variables
   public Vehicle(String ownerName, String vehicleName)
{
       this.ownerName = ownerName;
       this.vehicleName = vehicleName;

   }
   //method to update registration fees
   public static void updateRegistrationFee(double
registrationFee) {
       Vehicle.registrationFee = registrationFee;
   }

   //method to display vehicle details
   public void displayDetails() {
       System.out.println("owner name : " +
this.ownerName);
       System.out.println("vehicle name : " +
```

```
this.vehicleName);
        System.out.println("registration fees : " +
registrationFee);
    }

    //main method
    public static void main(String[] args) {

        //creating scanner class object to take user
input
        Scanner input = new Scanner(System.in);

        //taking user input for institute name
        System.out.print("Enter registration fees : ");
        double registrationFee = input.nextDouble();

        //updating the registration fees
        updateRegistrationFee(registrationFee);

        input.nextLine();
        //taking user input for vehicle details
        System.out.println("Enter details for vehicle "
+ 1);
        System.out.print("Enter owner : ");
        String ownerName = input.nextLine();
        System.out.print("Enter vehicle name : ");
        String vehicleName = input.nextLine();

        //creating object vehicle1 of Product class
```

```
        Vehicle vehicle1 = new Vehicle(ownerName,
vehicleName);

        //printing details of vehicle 1
        System.out.println("details of vehicle 1 : ");
        vehicle1.displayDetails();


        //taking user input for new registration fees
        System.out.print("Enter new registration fees :
");
        registrationFee = input.nextDouble();

        //updating the registration fees
        updateRegistrationFee(registrationFee);

        input.nextLine();
        //taking user input for vehicle 2
        System.out.println("Enter details for vehicle "
+ 2);
        System.out.print("Enter owner name : ");
        ownerName = input.nextLine();
        System.out.print("Enter vehicle name : ");
        vehicleName = input.nextLine();

        //creating object vehicle2 of vehicle class
        Vehicle vehicle2 = new Vehicle(ownerName,
vehicleName);
```

```
        //printing details of vehicle 2
        System.out.println("details of vehicle 2 : ");
        vehicle2.displayDetails();

        //printing details of vehicle 1 and you can see
registration fees is also updated for vehicle 1
        System.out.println("details of vehicle 1 : ");
        vehicle1.displayDetails();

        System.out.println("you can see above the
registration fees is also updated for vehicle 1");

        //closing the input
        input.close();
    }
}
```

---

2. Access Modifiers

Problem 1: University Management System

Create a `Student` class with:

- `rollNumber` (public).
- `name` (protected).
- `CGPA` (private).

Write methods to:

- Access and modify `CGPA` using public methods.

- Create a subclass `PostgraduateStudent` to demonstrate the use of protected members.

```
class Student {
    // Public field
    public int rollNumber;

    // Protected field
    protected String name;

    // Private field
    private double CGPA;

    // Constructor to initialize the values
    public Student(int rollNumber, String name, double
CGPA) {
        this.rollNumber = rollNumber;
        this.name = name;
        this.CGPA = CGPA;
    }

    // Public method to get the CGPA
    public double getCGPA() {
        return CGPA;
    }

    // Public method to set the CGPA
    public void setCGPA(double CGPA) {
        if (CGPA >= 0 && CGPA <= 10) {
            this.CGPA = CGPA;
```

```
        } else {
            System.out.println("Invalid CGPA value.
Please enter a value between 0 and 10.");
        }
    }

    // Method to display Student information
    public void displayStudentInfo() {
        System.out.println("Roll Number: " +
rollNumber);
        System.out.println("Name: " + name);
        System.out.println("CGPA: " + CGPA);
    }
}

// PostgraduateStudent
class PostgraduateStudent extends Student {

    // Constructor to initialize the values for
PostgraduateStudent
    public PostgraduateStudent(int rollNumber, String
name, double CGPA) {
        super(rollNumber, name, CGPA);
    }

    // Method to display PostgraduateStudent's specific
information
    public void displayPostgraduateInfo() {
        // Accessing the protected 'name' from the
```

```
superclass (Student)
        System.out.println("Postgraduate Student
Info:");
        System.out.println("Name: " + name);  //
Protected member is accessible here
        displayStudentInfo();  // Calling the method of
the base class
    }
}

public class StudentDetails {
    public static void main(String[] args) {
        // Creating a Student object
        Student student = new Student(101, "John Doe",
8.5);
        student.displayStudentInfo();

        // Modifying CGPA using setter method
        student.setCGPA(9.0);
        System.out.println("Updated CGPA: " +
student.getCGPA());

        // Creating a PostgraduateStudent object
        PostgraduateStudent postgradStudent = new
PostgraduateStudent(201, "Alice Smith", 9.2);
        postgradStudent.displayPostgraduateInfo();
    }
}
```

Problem 2: Book Library System

Design a `Book` class with:

- `ISBN` (public).
- `title` (protected).
- `author` (private).

Write methods to:

- Set and get the `author` name.
- Create a subclass `EBook` to access `ISBN` and `title` and demonstrate access modifiers.

```
class Book3 {
    // Public field for ISBN
    public String ISBN;

    // Protected field for title
    protected String title;

    // Private field for author
    private String author;

    // Constructor to initialize the Book object
    public Book3(String ISBN, String title, String
author) {
        this.ISBN = ISBN;
        this.title = title;
        this.author = author;
    }
```

```
    // Public method to get the author
    public String getAuthor() {
        return author;
    }

    // Public method to set the author
    public void setAuthor(String author) {
        this.author = author;
    }

    // Method to display book information
    public void displayBookInfo() {
        System.out.println("ISBN: " + ISBN);
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
    }
}

class EBook extends Book3 {

    // Constructor to initialize the EBook object
    public EBook(String ISBN, String title, String
author) {
        super(ISBN, title, author);
    }

    // Method to display EBook-specific information
    public void displayEBookInfo() {
```

```
        System.out.println("EBook Info:");
        System.out.println("ISBN: " + ISBN);  //
Accessing public field (ISBN)
        System.out.println("Title: " + title);  //
Accessing protected field (title)
        // Cannot directly access 'author' here because
it's private in the parent class
        System.out.println("Author: " + getAuthor());
// Using the public method to get the author's name
    }
}

// Library2 class (to test the implementation)
public class Library2 {
    public static void main(String[] args) {
        // Creating a Book object
        Book3 book = new Book3("123-456-789",
"Introduction to Java", "John Smith");
        book.displayBookInfo();

        // Modifying author using setter method
        book.setAuthor("Jane Doe");
        System.out.println("Updated Author: " +
book.getAuthor());

        // Creating an EBook object
        EBook eBook = new EBook("987-654-321", "Advanced
Java Programming", "Alice Brown");
        eBook.displayEBookInfo();
```

```
    }
}
```

Problem 3: Bank Account Management

Create a `BankAccount` class with:

- `accountNumber` (public).
- `accountHolder` (protected).
- `balance` (private).

Write methods to:

- Access and modify `balance` using public methods.
- Create a subclass `SavingsAccount` to demonstrate access to `accountNumber` and `accountHolder`.

```
// BankAccount class
class BankAccount {
    // Public field for account number
    public String accountNumber;

    // Protected field for account holder's name
    protected String accountHolder;

    // Private field for balance
    private double balance;

    // Constructor to initialize the BankAccount object
    public BankAccount(String accountNumber, String
```

```
accountHolder, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    // Public method to get the balance
    public double getBalance() {
        return balance;
    }

    // Public method to set the balance
    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Balance cannot be
negative.");
        }
    }

    // Method to deposit money
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Deposit amount must be
positive.");
```

```
        }
    }

    // Method to withdraw money
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Invalid withdrawal
amount.");
        }
    }

    // Method to display account details
    public void displayAccountInfo() {
        System.out.println("Account Number: " +
accountNumber);
        System.out.println("Account Holder: " +
accountHolder);
        System.out.println("Balance: " + balance);
    }
}

// SavingsAccount class
class SavingsAccount extends BankAccount {

    // Constructor to initialize the SavingsAccount
object
```

```
    public SavingsAccount(String accountNumber, String
accountHolder, double balance) {
        super(accountNumber, accountHolder, balance);
    }

    // Method to display SavingsAccount-specific
information
    public void displaySavingsAccountInfo() {
        System.out.println("Savings Account Info:");
        System.out.println("Account Number: " +
accountNumber); // Accessing public field
(accountNumber)
        System.out.println("Account Holder: " +
accountHolder); // Accessing protected field
(accountHolder)
        displayAccountInfo(); // Calling the method from
the base class to display full account details
    }
}

//ATM class (to test the implementation)
public class ATM {
    public static void main(String[] args) {
        // Creating a BankAccount object
        BankAccount bankAccount = new
BankAccount("1234567890", "John Doe", 5000.0);
        bankAccount.displayAccountInfo();

        // Modifying balance using setter method
```

```
        bankAccount.setBalance(6000.0);
        System.out.println("Updated Balance: " +
bankAccount.getBalance());

        // Depositing money
        bankAccount.deposit(1000.0);
        System.out.println("New Balance after Deposit: "
+ bankAccount.getBalance());

        // Withdrawing money
        bankAccount.withdraw(1500.0);
        System.out.println("New Balance after
Withdrawal: " + bankAccount.getBalance());

        // Creating a SavingsAccount object
        SavingsAccount savingsAccount = new
SavingsAccount("9876543210", "Alice Smith", 10000.0);
        savingsAccount.displaySavingsAccountInfo();
    }
}
```

Problem 4: Employee Records

Develop an `Employee` class with:

- `employeeID` (public).
- `department` (protected).
- `salary` (private).

CodInClub

Powered By BridgeLabz

Write methods to:

- Modify `salary` using a public method.
- Create a subclass `Manager` to access `employeeID` and `department`.

```
// Employee class
class Employee {
    // Public field for employee ID
    public int employeeID;

    // Protected field for department
    protected String department;

    // Private field for salary
    private double salary;

    // Constructor to initialize the Employee object
    public Employee(int employeeID, String department,
double salary) {
        this.employeeID = employeeID;
        this.department = department;
        this.salary = salary;
    }

    // Public method to get the salary
    public double getSalary() {
        return salary;
    }

    // Public method to modify the salary
```

```
    public void setSalary(double salary) {
        if (salary >= 0) {
            this.salary = salary;
        } else {
            System.out.println("Salary cannot be
negative.");
        }
    }

    // Method to display employee information
    public void displayEmployeeInfo() {
        System.out.println("Employee ID: " +
employeeID);
        System.out.println("Department: " + department);
        System.out.println("Salary: " + salary);
    }
}

// Manager class
class Manager extends Employee {

    // Constructor to initialize the Manager object
    public Manager(int employeeID, String department,
double salary) {
        super(employeeID, department, salary);
    }

    // Method to display Manager-specific information
    public void displayManagerInfo() {
```

```
        System.out.println("Manager Info:");
        System.out.println("Employee ID: " +
employeeID);  // Accessing public field (employeeID)
        System.out.println("Department: " + department);
// Accessing protected field (department)
        displayEmployeeInfo();  // Calling the method
from the base class to display full employee details
    }
}

// EmployeeDetails class (to test the implementation)
public class EmployeeDetails {
    public static void main(String[] args) {
        // Creating an Employee object
        Employee employee = new Employee(101, "Finance",
50000.0);
        employee.displayEmployeeInfo();

        // Modifying salary using setter method
        employee.setSalary(55000.0);
        System.out.println("Updated Salary: " +
employee.getSalary());

        // Creating a Manager object
        Manager manager = new Manager(102, "HR",
75000.0);
        manager.displayManagerInfo();
    }
}
```

CodInClub

Powered By BridgeLabz